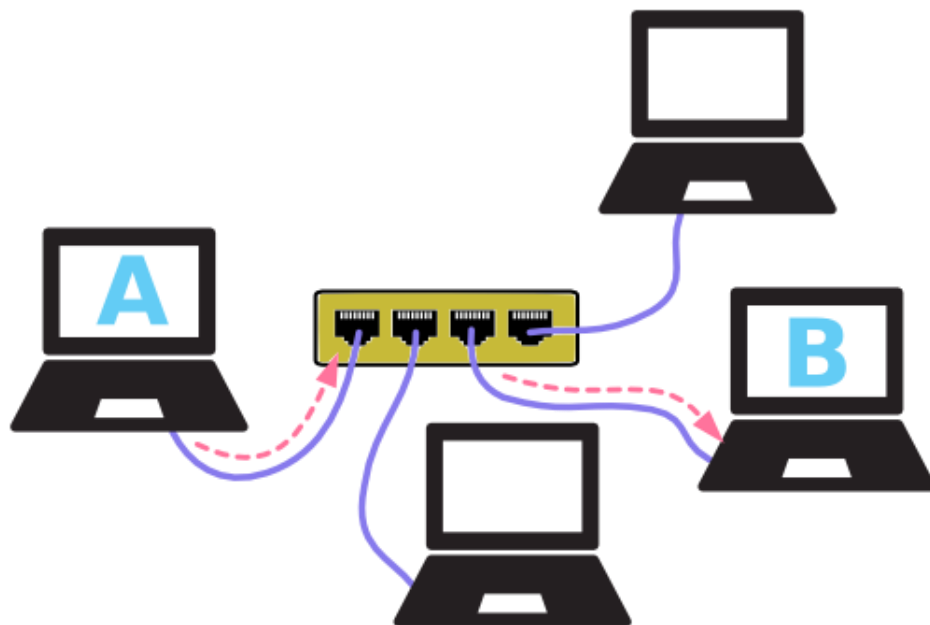


COMP2190 SOCKET PROGRAMMING

PROJECT DOCUMENTATION



Name: Cajaan Campbell

ID: 620169526

Date: September 30, 2025

Program Design

This project implements a client–server protocol in Python 3 using sockets. The communication simulates aspects of a Diffie–Hellman key exchange combined with a zero knowledge style challenge response for liveness.

Server:

- Accepts user input for a prime number “p” and a valid generator “g”.
- Generates its private key “x”, computes public key $g^x \bmod p$.
- Responds to client hello, sends generator + prime, exchanges public keys, and computes shared secret.
- Issues a nonce challenge, verifies the client’s response, and replies with a success or error code.

Client:

- Sends a hello message to initiate communication.
- Receives generator and prime, generates private key “y”, computes public key $g^y \bmod p$.
- Exchanges public keys with the server, computes the shared secret.
- Transforms the server’s nonce (subtracts 5) and returns the result.
- Verifies server’s confirmation.

Both sides print informative messages at each step so execution can be followed, and debugging is easy.

How it works

1) Handshake:

- Client → Server: "100 Hello"
- Server → Client: "105 Generator + Prime g p "

2) Key Exchange:

- Client generates private " y ", sends "120 PubKey $g^y \bmod p$."
- Server generates private " x ", sends "121 PubKey $g^x \bmod p$ ".
- Both compute the shared secret " $g^{(xy)} \bmod p$ ".

3) Nonce Challenge:

- Server sends random nonce: "130 Nonce <value>".
- Client transforms nonce (subtracts 5), sends "131 Transformed Nonce <value-5>".

4) Verification:

- If the client's transformed nonce matches expectation → Server replies "220 Verified".
- Otherwise → Server replies "400 Error".

Design Tradeoffs

Validation:

- Primes and generators are validated to prevent invalid parameters.

Nonce transformation:

- Subtracting 5 is a simple, demonstrable transformation to mimic a proof of knowledge, while avoiding revealing private keys.

Socket management:

- One persistent socket is used per session (cleaner and more efficient).

Randomness:

- SystemRandom ensures cryptographically strong randomness.

Testing

The following tests were conducted:

- **Basic run (small prime example):**

Client and server executed on localhost using the pair ($g=5$, $p=23$). The handshake completed, public keys were exchanged, a shared secret was derived, and nonce verification returned “verification successful”.

- **Larger prime/generator run:**

The pair ($g=11$, $p=769$) was entered. The client successfully received the server’s public key and nonce, computed the shared secret, and verified the response. Output values differed from the smaller prime test, confirming correct handling of larger parameters.

- **Testing with multiple valid pairs:**

Additional runs were performed with the valid (g , p) pairs provided:

(5,47),(19,83),(3,257),(19,719),(5,743),(5,907). In each case, the server generated public information, the client computed the shared secret, and nonce verification succeeded.

- **Consistency across runs:**

Repeated executions with the same (g , p) pair produced different numeric outputs for public keys and nonces, due to random private key selection, but the protocol consistently verified successfully.

Testing Results:

1) Pair (5,23) (Server)

```
● cajaun@Mac project-1 % python3 server.py 5060
Enter server name: Cajaun
Server of Cajaun
Enter a prime number between 257 and 4093: 23
Enter a generator for the prime number: 5
Listening on port 5060...
Connected from: ('127.0.0.1', 60570)
Received: 100 Hello
Received: 120 PubKey 8
Received: 131 Transformed Nonce 11

--- SERVER SUMMARY ---
Server Name = Cajaun
g = 5
p = 23
Server private key (x) = 18
Server public key = 6
Client public key = 8
Shared secret = 12
Nonce = 16
-----
```

1.5) Pair (5,23) (Client)

```
● cajaan@Mac project-1 % python3 client.py localhost 5060
Enter client name: Cajaan
Client of Cajaan
```

The purpose of this program is to implement a simple program that will carry out an exchange of messages that contains elements of a zero knowledge proof. The client sends a hello message to the server. The server responds with a message containing a generator, a prime, some public information, and a commitment message. The client will respond with a challenge message that contains an integer. The server transforms that received integer and then sends a response message to the client. The client checks that this transformed integer shows that the server knows the secret (without the secret being revealed).

```
Received: 105 Generator + Prime 5 23
```

```
Received: 121 PubKey 6
```

```
Shared secret computed.
```

```
Received: 130 Nonce 16
```

```
Received: 220 Verified
```

```
Verification successful!
```

```
--- CLIENT SUMMARY ---
```

```
Client Name = Cajaan
```

```
g = 5
```

```
p = 23
```

```
Client private key (y) = 6
```

```
Client public key = 8
```

```
Server public key = 6
```

```
Shared secret = 12
```

```
Nonce = 16 | Transformed = 11
```

```
-----
```

2) Pair (5, 47) (Server)

```
● cajaan@Mac project-1 % python3 server.py 5060
Enter server name: Cajaan
Server of Cajaan
Enter a prime number between 257 and 4093: 47
Enter a generator for the prime number: 5
Listening on port 5060...
Connected from: ('127.0.0.1', 60605)
Received: 100 Hello
Received: 120 PubKey 14
Received: 131 Transformed Nonce 9

--- SERVER SUMMARY ---
Server Name = Cajaan
g = 5
p = 47
Server private key (x) = 14
Server public key = 27
Client public key = 14
Shared secret = 12
Nonce = 14
-----
```


2.5) Pair (5, 47) (Client)

```
● cajaun@Mac project-1 % python3 client.py localhost 5060
```

```
Enter client name: Cajaun
```

```
Client of Cajaun
```

The purpose of this program is to implement a simple program that will carry out an exchange of messages that contains elements of a zero knowledge proof. The client sends a hello message to the server. The server responds with a message containing a generator, a prime, some public information, and a commitment message. The client will respond with a challenge message that contains an integer. The server transforms that received integer and then sends a response message to the client. The client checks that this transformed integer shows that the server knows the secret (without the secret being revealed).

```
Received: 105 Generator + Prime 5 47
```

```
Received: 121 PubKey 27
```

```
Shared secret computed.
```

```
Received: 130 Nonce 14
```

```
Received: 220 Verified
```

```
Verification successful!
```

```
--- CLIENT SUMMARY ---
```

```
Client Name = Cajaun
```

```
g = 5
```

```
p = 47
```

```
Client private key (y) = 4
```

```
Client public key = 14
```

```
Server public key = 27
```

```
Shared secret = 12
```

```
Nonce = 14 | Transformed = 9
```

```
-----
```

3) Pair (19, 83) (Server)

```
● cajaun@Mac project-1 % python3 server.py 5060
Enter server name: Cajaun
Server of Cajaun
Enter a prime number between 257 and 4093: 83
Enter a generator for the prime number: 19
Listening on port 5060...
Connected from: ('127.0.0.1', 60651)
Received: 100 Hello
Received: 120 PubKey 76
Received: 131 Transformed Nonce 30

--- SERVER SUMMARY ---
Server Name = Cajaun
g = 19
p = 83
Server private key (x) = 46
Server public key = 40
Client public key = 76
Shared secret = 41
Nonce = 35
-----
```

3.5) Pair (19, 83) (Client)

```
● cajaun@Mac project-1 % python3 client.py localhost 5060
Enter client name: Cajaun
Client of Cajaun
```

The purpose of this program is to implement a simple program that will carry out an exchange of messages that contains elements of a zero knowledge proof. The client sends a hello message to the server. The server responds with a message containing a generator, a prime, some public information, and a commitment message. The client will respond with a challenge message that contains an integer. The server transforms that received integer and then sends a response message to the client. The client checks that this transformed integer shows that the server knows the secret (without the secret being revealed).

```
Received: 105 Generator + Prime 19 83
```

```
Received: 121 PubKey 40
```

```
Shared secret computed.
```

```
Received: 130 Nonce 35
```

```
Received: 220 Verified
```

```
Verification successful!
```

```
--- CLIENT SUMMARY ---
```

```
Client Name = Cajaun
```

```
g = 19
```

```
p = 83
```

```
Client private key (y) = 15
```

```
Client public key = 76
```

```
Server public key = 40
```

```
Shared secret = 41
```

```
Nonce = 35 | Transformed = 30
```

```
-----
```

4) Pair (3, 257) (Server)

```
● cajaan@Mac project-1 % python3 server.py 5060
Enter server name: Cajaan
Server of Cajaan
Enter a prime number between 257 and 4093: 257
Enter a generator for the prime number: 3
Listening on port 5060...
Connected from: ('127.0.0.1', 60671)
Received: 100 Hello
Received: 120 PubKey 128
Received: 131 Transformed Nonce 166

--- SERVER SUMMARY ---
Server Name = Cajaan
g = 3
p = 257
Server private key (x) = 127
Server public key = 171
Client public key = 128
Shared secret = 255
Nonce = 171
-----
```

4.5) Pair (3, 257) (Client)

```
● cajaan@Mac project-1 % python3 client.py localhost 5060
Enter client name: Cajaan
Client of Cajaan
```

The purpose of this program is to implement a simple program that will carry out an exchange of messages that contains elements of a zero knowledge proof. The client sends a hello message to the server. The server responds with a message containing a generator, a prime, some public information, and a commitment message. The client will respond with a challenge message that contains an integer. The server transforms that received integer and then sends a response message to the client. The client checks that this transformed integer shows that the server knows the secret (without the secret being revealed).

```
Received: 105 Generator + Prime 3 257
```

```
Received: 121 PubKey 171
```

```
Shared secret computed.
```

```
Received: 130 Nonce 171
```

```
Received: 220 Verified
```

```
Verification successful!
```

```
--- CLIENT SUMMARY ---
```

```
Client Name = Cajaan
```

```
g = 3
```

```
p = 257
```

```
Client private key (y) = 80
```

```
Client public key = 128
```

```
Server public key = 171
```

```
Shared secret = 255
```

```
Nonce = 171 | Transformed = 166
```

```
-----
```

5) Pair (19, 719) (Server)

```
● cajaun@Mac project-1 % python3 server.py 5060
Enter server name: Cajaun
Server of Cajaun
Enter a prime number between 257 and 4093: 719
Enter a generator for the prime number: 19
Listening on port 5060...
Connected from: ('127.0.0.1', 60693)
Received: 100 Hello
Received: 120 PubKey 73
Received: 131 Transformed Nonce 317

--- SERVER SUMMARY ---
Server Name = Cajaun
g = 19
p = 719
Server private key (x) = 244
Server public key = 627
Client public key = 73
Shared secret = 100
Nonce = 322
-----
```

5.5) Pair (19, 719) (Server)

```
● cajaun@Mac project-1 % python3 client.py localhost 5060
Enter client name: Cajaun
Client of Cajaun

The purpose of this program is to implement a simple program that will carry
out an exchange of messages that contains elements of a zero knowledge proof.
The client sends a hello message to the server. The server responds with a
message containing a generator, a prime, some public information, and
a commitment message. The client will respond with a challenge message that
contains an integer. The server transforms that received integer and then
sends a response message to the client. The client checks that this
transformed integer shows that the server knows the secret (without the
secret being revealed).
Received: 105 Generator + Prime 19 719
Received: 121 PubKey 627
Shared secret computed.
Received: 130 Nonce 322
Received: 220 Verified
Verification successful!

--- CLIENT SUMMARY ---
Client Name = Cajaun
g = 19
p = 719
Client private key (y) = 435
Client public key = 73
Server public key = 627
Shared secret = 100
Nonce = 322 | Transformed = 317
-----
```

6) Pair (5, 743) (Server)

```
● cajaun@Mac project-1 % python3 server.py 5060
Enter server name: Cajaun
Server of Cajaun
Enter a prime number between 257 and 4093: 743
Enter a generator for the prime number: 5
Listening on port 5060...
Connected from: ('127.0.0.1', 60710)
Received: 100 Hello
Received: 120 PubKey 296
Received: 131 Transformed Nonce 377

--- SERVER SUMMARY ---
Server Name = Cajaun
g = 5
p = 743
Server private key (x) = 502
Server public key = 130
Client public key = 296
Shared secret = 85
Nonce = 382
-----
```


6.5) Pair (5, 743) (Client)

```
● cajaun@Mac project-1 % python3 client.py localhost 5060
Enter client name: Cajaun
Client of Cajaun
```

The purpose of this program is to implement a simple program that will carry out an exchange of messages that contains elements of a zero knowledge proof. The client sends a hello message to the server. The server responds with a message containing a generator, a prime, some public information, and a commitment message. The client will respond with a challenge message that contains an integer. The server transforms that received integer and then sends a response message to the client. The client checks that this transformed integer shows that the server knows the secret (without the secret being revealed).

```
Received: 105 Generator + Prime 5 743
```

```
Received: 121 PubKey 130
```

```
Shared secret computed.
```

```
Received: 130 Nonce 382
```

```
Received: 220 Verified
```

```
Verification successful!
```

```
--- CLIENT SUMMARY ---
```

```
Client Name = Cajaun
```

```
g = 5
```

```
p = 743
```

```
Client private key (y) = 680
```

```
Client public key = 296
```

```
Server public key = 130
```

```
Shared secret = 85
```

```
Nonce = 382 | Transformed = 377
```

```
-----
```

7) Pair (11, 769) (Server)

```
● cajaan@Mac project-1 % python3 server.py 5060
Enter server name: Cajaan
Server of Cajaan
Enter a prime number between 257 and 4093: 769
Enter a generator for the prime number: 11
Listening on port 5060...
Connected from: ('127.0.0.1', 60773)
Received: 100 Hello
Received: 120 PubKey 495
Received: 131 Transformed Nonce 297

--- SERVER SUMMARY ---
Server Name = Cajaan
g = 11
p = 769
Server private key (x) = 572
Server public key = 203
Client public key = 495
Shared secret = 322
Nonce = 302
-----
```

7.5) Pair (11, 769) (Client)

```
● cajaan@Mac project-1 % python3 client.py localhost 5060
```

```
Enter client name: Cajaan
```

```
Client of Cajaan
```

The purpose of this program is to implement a simple program that will carry out an exchange of messages that contains elements of a zero knowledge proof. The client sends a hello message to the server. The server responds with a message containing a generator, a prime, some public information, and a commitment message. The client will respond with a challenge message that contains an integer. The server transforms that received integer and then sends a response message to the client. The client checks that this transformed integer shows that the server knows the secret (without the secret being revealed).

```
Received: 105 Generator + Prime 11 769
```

```
Received: 121 PubKey 203
```

```
Shared secret computed.
```

```
Received: 130 Nonce 302
```

```
Received: 220 Verified
```

```
Verification successful!
```

```
--- CLIENT SUMMARY ---
```

```
Client Name = Cajaan
```

```
g = 11
```

```
p = 769
```

```
Client private key (y) = 191
```

```
Client public key = 495
```

```
Server public key = 203
```

```
Shared secret = 322
```

```
Nonce = 302 | Transformed = 297
```

```
-----
```

8) Pair (5, 907) (Server)

```
● cajaun@Mac project-1 % python3 server.py 5060
Enter server name: Cajaun
Server of Cajaun
Enter a prime number between 257 and 4093: 907
Enter a generator for the prime number: 5
Listening on port 5060...
Connected from: ('127.0.0.1', 60823)
Received: 100 Hello
Received: 120 PubKey 447
Received: 131 Transformed Nonce 603

--- SERVER SUMMARY ---
Server Name = Cajaun
g = 5
p = 907
Server private key (x) = 848
Server public key = 721
Client public key = 447
Shared secret = 900
Nonce = 608
-----
```

8.5) Pair (5, 907) (Client)

```
● cajaun@Mac project-1 % python3 client.py localhost 5060
Enter client name: Cajaun
Client of Cajaun

The purpose of this program is to implement a simple program that will carry
out an exchange of messages that contains elements of a zero knowledge proof.
The client sends a hello message to the server. The server responds with a
message containing a generator, a prime, some public information, and
a commitment message. The client will respond with a challenge message that
contains an integer. The server transforms that received integer and then
sends a response message to the client. The client checks that this
transformed integer shows that the server knows the secret (without the
secret being revealed).
Received: 105 Generator + Prime 5 907
Received: 121 PubKey 721
Shared secret computed.
Received: 130 Nonce 608
Received: 220 Verified
Verification successful!

--- CLIENT SUMMARY ---
Client Name = Cajaun
g = 5
p = 907
Client private key (y) = 140
Client public key = 447
Server public key = 721
Shared secret = 900
Nonce = 608 | Transformed = 603
-----
```

9) Invalid input

```
○ cajaun@Mac project-1 % python3 server.py 5060
Enter server name: Cajaun
Server of Cajaun
Enter a prime number between 257 and 4093: 1000
Number is not prime.
Enter a prime number between 257 and 4093: █
```

Interoperability Test

To simulate real-world interoperability:

- The client was run against another student's server implementation.
- The server was run against another student's client implementation.
- Both sides successfully completed the key exchange and nonce verification.

Interoperability Results

1) Server connected to classmate's client, exchange completed.

```
● cajaan@cajajuns-MacBook project-1 % python3 server.py 5001

Enter server name: cajaan
Server of cajaan
Enter a prime number between 257 and 4093: 719
Enter a generator for the prime number: 19
Listening on port 5001...
Connected from: ('10.86.77.80', 51765)
Received: 100 Hello
Received: 120 PubKey 647
Received: 131 Transformed Nonce 518

--- SERVER SUMMARY ---
Server Name = cajaan
g = 19
p = 719
Server private key (x) = 386
Server public key = 145
Client public key = 647
Shared secret = 26
Nonce = 523
-----
○ cajaan@cajajuns-MacBook project-1 % █
```

```
PS C:\Users\hackz\Downloads\starter_code\starter_code> python server.py 5001
Server of Kishu Johnally
Starting server and waiting for client connection...
Enter a prime number between 257 and 4093: 257
Enter a generator (g < 257): 3
Using generator g=3, prime p=257
Generated server private key x: 106
Computed server public key  $g^x \bmod p$ : 13
Listening on port 5001...
Connected from: ('10.86.77.168', 56797)
Received from client: 100 Hello
Sending generator and prime: 105 Generator + Prime 3 257
Received from client: 120 PubKey 161
Client public key received: 161
Sending server public key: 121 PubKey 13
Generated nonce: 180
Sending nonce: 130 Nonce 180
Received from client: 131 Transformed Nonce 175
Transformed nonce received: 175
Sending verification: 220 Verified
g: 3
p: 257
Server private key: 106
Server public key: 13
Client public key: 161
Shared secret: 52
Verified: True
Closing connection and terminating server.
PS C:\Users\hackz\Downloads\starter_code\starter_code> |
```


2) Client connected to classmate's server, exchange completed.

```
● cajaan@cajajuns-MacBook project-1 % python3 client.py 10.86.77.80 5001
Enter client name: cajaan
Client of cajaan

The purpose of this program is to implement a simple program that will carry
out an exchange of messages that contains elements of a zero knowledge proof.
The client sends a hello message to the server. The server responds with a
message containing a generator, a prime, some public information, and
a commitment message. The client will respond with a challenge message that
contains an integer. The server transforms that received integer and then
sends a response message to the client. The client checks that this
transformed integer shows that the server knows the secret (without the
secret being revealed).
Received: 105 Generator + Prime 3 257
Received: 121 PubKey 13
Shared secret computed.
Received: 130 Nonce 180
Received: 220 Verified
Verification successful!

--- CLIENT SUMMARY ---
Client Name = cajaan
g = 3
p = 257
Client private key (y) = 113
Client public key = 161
Server public key = 13
Shared secret = 52
Nonce = 180 | Transformed = 175
-----
○ cajaan@cajajuns-MacBook project-1 %
```


Client of Kishu Johnally

Starting client and preparing to connect to server...

The purpose of this program is to implement a simple program that will carry out an exchange of messages that contains elements of a zero knowledge proof. The client sends a hello message to the server. The server responds with a message containing a generator, a prime, some public information, and a commitment message. The client will respond with a challenge message that contains an integer. The server transforms that received integer and then sends a response message to the client. The client checks that this transformed integer shows that the server knows the secret (without the secret being revealed).

Connecting to server at 10.86.77.168:5001...

Connected to server.

Sending hello message to server: 100 Hello

Received from server: 105 Generator + Prime 19 719

Generator received: g=19, Prime received: p=719

Generated client private key y: 421

Computed client public key $g^y \bmod p$: 647

Sending client public key: 120 PubKey 647

Received from server: 121 PubKey 145

Server public key received: 145

Received from server: 130 Nonce 523

Nonce received from server: 523

Sending transformed nonce: 131 Transformed Nonce 518

Received from server: 220 Verified

Server verified the transformation.

g: 19

p: 719

Client private key: 421

Client public key: 647

Server public key: 145

Shared secret: 26

Verified: True

Closing socket and terminating client.

PS C:\Users\hackz\Downloads\starter code\starter code>

Limitations & Improvements

Prime checking

- Currently handled with helper functions. Could use probabilistic primality tests (e.g., Miller–Rabin) for large primes.

Error handling

- Could add richer status codes (e.g., “410 Invalid Input”) for better debugging.

Security extension

- Replace the simple “subtract 5” nonce check with a cryptographic hash involving the shared secret.

Concurrency

- Current server handles only one client; could be extended to multi-client support using threads or asyncio.

Conclusion

The implementation demonstrates secure message exchange principles (Diffie–Hellman + nonce-based verification) in a clear and testable way. It is modular, debuggable, and extensible for future improvements such as stronger security primitives and multi-client scalability.